

Ward: Phase 0 Execution Plan & Design Reference

Compiled from design documentation and planning conversations

Status: Phase 0 — Minimal Falsification Experiment

Date: 2026-07-31

Version: 1.0 — Compiled Planning Document

1. Executive Summary

Ward is a **standalone verification-first programming language** purpose-built for AI-authored full-stack software development. It is not an addon, toolkit, or extension of an existing language. It is a complete compiler toolchain with its own surface grammar, core calculus, elaborator, verifier, and multi-target backend.

This document compiles the complete design rationale, honest feasibility assessment, and concrete Phase 0 execution plan derived from the Ward Language Design document and Ward Research Proposal.

Central Hypothesis: A familiar Python/TypeScript-shaped surface syntax that elaborates into a strict, dependently-typed core — combined with tiered verification, a trusted FFI boundary for unverified libraries, and a full-stack effect model — will produce higher converged accuracy on realistic coding tasks than existing verification-first languages (Dafny, Lean 4, SPARK/Ada, Verus) when used with AI code generation.

Phase 0 Goal: Falsify or validate the two highest-leverage, cheapest-to-test claims before any language construction begins:

1. Surface-syntax elaboration outperforms direct formal-syntax generation.
 2. A contract-stub trust boundary catches more errors at library call sites than unverified direct calls.
-

2. Design Philosophy

2.1 Three Resolved False Binaries

1. **“Familiar syntax vs. strict semantics”** → Resolved by splitting the language into a **Surface Layer** (what the model writes, familiar) and a **Core Calculus** (what gets checked and executed, strict).
2. **“Make the model smarter vs. make errors easier to catch”** → Resolved by reducing how much novel code the model must write via a **global, proof-verified, content-addressed library** that composition is drawn from by default.
3. **“Verify everything vs. verify nothing”** → Resolved by **tiered verification**: most full-stack code is contract-checked and tested; only consequential logic gets full proof.

2.2 Governing Principle

“Compiles” means “provably satisfies its stated contract, at whatever tier that contract requires” — not “type-checks and looks plausible,” and not “everything must be a theorem.”

2.3 Why Standalone (Not an Addon)

The following features require language-level control over the entire compilation pipeline and cannot be bolted onto Python, TypeScript, Dafny, or Verus without fundamentally rebuilding the host:

Feature	Why It Requires a New Language
Surface/Core Split	Needs a Python/TS-shaped surface that elaborates into a strict dependent/refinement-typed core. Cannot be added to Dafny/Lean without replacing their syntax; cannot be added to Python because its dynamic semantics cannot support verification.
Tiered Verification	Compile-mode distinction (Tested / Contracted / Proven) enforced by the compiler and type system, not a library or linter setting.
Mandatory <code>trust</code>: FFI Boundary	First-class language construct changing what the verifier attempts to prove at the boundary. Requires the compiler to treat external calls as opaque stubs with stated contracts.
Full-Stack Effect Vocabulary	Tracked in the type system at the function signature level (<code>net</code> , <code>async</code> , <code>render</code> , <code>db</code>). Cannot be added to Python/JS without a new type checker; Dafny's existing model is heap-only.
Grammar-Constrained + Type-Constrained Decoding	Requires owning the grammar specification and decoder constraints end-to-end.
Content-Addressed Composition Registry	Functions identified by hash of content + proven contract. Changes how the linker, package manager, and compiler resolve symbols.
Multi-Target Compilation	Compiling one verified source to JS, Go, Python, JVM, WASM, and native binaries requires owning the backend pipeline.

2.4 Honest Assessment: Is This Possible?

Verdict: Technically achievable as a research artifact and probably achievable as a niche production tool (Dafny/SPARK scale: ~2,700–8,500 GitHub stars, institutional adopters). Almost certainly **not** achievable as a mainstream language.

Milestone	Probability
Phase 0 produces positive, publishable results	~70%
Phases 1–3 produce working compiler/checker for real patterns	~50%
Phase 4 (checker-driven RL for general software) works well enough	~20–30%
Ward reaches modest adoption target (Dafny/SPARK scale) within 5 years	~15–25%

What Gives Confidence: Every component is proven somewhere. The problem (SWE-bench Pro

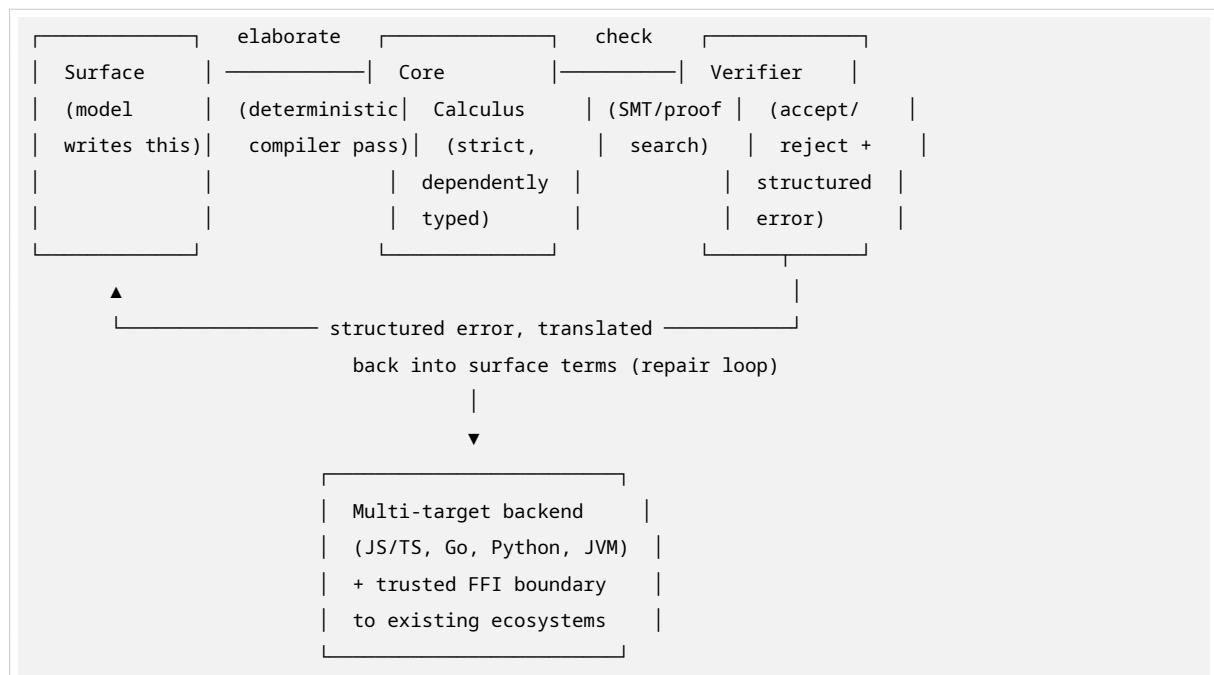
~45–70%, GitChameleon ~48–51%) is real and worsening. The phased structure is genuinely de-risked.

What Worries Most:

- The **build-vs-extend question** (Verus already has familiar syntax, native contracts, real ecosystem, and AI proof tools). Unless Phase 0 demonstrates that Rust-shaped syntax imposes a hard ceiling, the rational move may be to extend Verus.
- The “**trust gap**” may swallow the value proposition: a typical full-stack app depends on hundreds of unverified libraries. Ward could end up 90% `trust:` annotations.
- **Phase 4 is a moonshot**: checker-driven RL works in math (Lean 4) but has no evidence it generalizes to messy, stateful, real-world software.
- The **elaborator is a new bug surface**: mis-translation from familiar surface to strict core becomes a silent logic error that the verifier happily proves correct.

Recommended Optimization Target: Do not aim to surpass Dafny/Lean/SPARK. Aim to become the **default verification language for AI-generated full-stack code** — a narrower, more defensible target.

3. Architecture Overview



3.1 Layer 1: Surface (What the AI Writes)

- **Syntax:** Deliberately unoriginal. Python/TypeScript-shaped control flow, literals, function definitions.
- **Contracts:** Native `requires` / `ensures` / `old()` syntax (Eiffel-style, mandatory). The model states intent in plain terms; never writes dependent types or ownership annotations.
- **Grammar-Constrained Decoding:** Decoder samples only tokens valid for the surface grammar (XGrammar, Outlines, vLLM/SGLang). Eliminates syntax errors before logic is considered.

- **Confidence Typing:** Per-unit annotation (`@confidence: 0.92`) derived from token probabilities. Low-confidence units routed to heavier verification automatically.

3.2 Layer 2: Elaboration (Deterministic Bridge)

Not AI. A conventional, auditable compiler pass.

Responsibilities:

1. Expand `requires/ensures` into core-level refinement predicates.
2. Resolve all implicit coercions, numeric widening, type ambiguities into explicit core terms.
3. Infer effect annotations (`net`, `async`, `render`, `db`, `fs`, `mut`) by static analysis.
4. Infer and insert linear/ownership annotations for linear-typed capabilities.
5. Resolve dependencies against pinned version ranges — version mismatch is a compile-time error.

Hard Rule: If elaboration cannot resolve something unambiguously, it is a **hard error** surfaced immediately. The program never reaches the verifier in an ambiguous state.

3.3 Layer 3: Core Calculus (What Gets Checked)

- **Type System:** Refinement/dependent types (Liquid Haskell / F* lineage), SMT-backed.
- **Effect Tracking:** Koka-style, extended for full-stack: `net`, `async`, `render`, `db`, `fs`, `mut`, `partial`. Violating a declared effect is a compile-time error.
- **Linear/Ownership Types:** Applied only to consequential values (money, auth tokens, one-time capabilities). Inferred by elaborator — model never authors borrow annotations (fixing Rust’s 58–72% fluency penalty).
- **Totality:** Functions must be provably terminating unless explicitly marked `partial`. (Acknowledged: totality checkers are conservative and produce false positives.)

3.4 Layer 4: Verification & Repair Loop

- **Granular Obligations:** Functions decompose into small, independently provable units. Failures localize to (`location`, `violated-obligation`, `counterexample`) triples.
- **Structured Errors:** Never prose. Machine-parseable errors translated back into surface-language terms.
- **Adversarial Critic (CEGIS-style):** Second pass actively constructs counterexamples to the stated contract.
- **Property-Based Tests:** Generated mechanically from contracts (QuickCheck-style), never written by the model.

Tiered Enforcement:

Tier	Requirement	Typical Use
Tested	Property-based tests from type signature; no proof	UI glue, formatting, simple CRUD
Contracted	<code>requires/ensures</code> checked; bounded proof search (times out to test-based confidence)	Business logic, validation, most backend services

Table 3 – continued

Tier	Requirement	Typical Use
Proven	Full SMT verification; no timeout fallback; mandatory for linear-typed resources	Auth, payments, data integrity

3.5 Layer 5: Multi-Target Backend + Trusted FFI

- **Compilation Targets:** TypeScript/JavaScript (frontend), Go/Python/JVM/Node (backend), WebAssembly and native binaries (performance-critical).
- **Trusted FFI Boundary (Gradual Verification):**

```
extern fn stripe_charge(amount: Money, token: CardToken) -> Result<Charge, PaymentError>
  requires amount > 0
  ensures result.is_ok() implies result.unwrap().amount == amount
  trust: "stub — Stripe SDK v14.2, not independently verified"
```

- Inside the boundary, verifier reasons only about the *stated contract*, not implementation.
- **trust:** annotation is mandatory and machine-readable, producing an auditable trust report.
- **Limitation:** Proven-tier code must never target translation steps requiring probabilistic models (e.g., WASM → WGSL via SLM). Proven stays on deterministic backends only.

3.6 Layer 6: Composition-First Standard Library

- **Content-Addressed Registry:** Every function identified by hash of content + proven contract — not by name or file path.
- **Permanent Trust:** Once proof is checked, never needs re-verification.
- **Default Behavior:** Search-and-compose. Given a required contract, search for existing proven function. Writing new logic is the fallback.
- **Contract Templates:** Reusable, parameterized correctness patterns (conservation of value, no double-spend, monotonic counters, idempotent writes). **Safety Discipline:** Templates are *expansion sugar only* — they expand into a full, specific proof obligation that the SMT solver checks fresh every time. Never pre-proven shortcuts.

4. Checker Decision: Dafny for Phase 0

4.1 Why Dafny

Factor	Dafny	Verus
Published AI baseline	✅ 82–91% verify success, ~77% pass@1 ceiling documented	❌ No established AI-generation baseline
Off-distribution problem	✅ Documented specifically for Dafny formal syntax	❌ Not documented
Repair-loop infrastructure	✅ Reusable self-healing pipelines	⚠️ Less mature
Ecosystem gap relevance	✅ Irrelevant in Phase 0 (pure functions only)	N/A

Table 4 – continued

Factor	Dafny	Verus
Build-vs-extend analysis	✗ Not applicable	✓ Needed for Phase 0.5

Decision: Use **Dafny** as the Phase 0 primary checker. Verus is reserved for a **parallel spike in Weeks 9–12** (Phase 0.5) to answer the build-vs-extend question.

4.2 Verus Spike (Phase 0.5, Weeks 9–12)

Answer whether extending Verus is cheaper than building Ward standalone:

- Can we write a `ward0` → Verus transpiler in ~1 week?
- Does Verus’s borrow checker help or interfere with surface contract elaboration?
- How much of the FFI boundary can be simulated using Verus’s `unsafe` / external mechanisms?

If the spike shows extension is straightforward, pivot to a Verus frontend. If Verus’s ownership model blocks the desired surface syntax, that is evidence Ward must be standalone.

5. Phase 0: Minimal Falsification Experiment

5.1 Meta

Attribute	Value
Duration	~3 months
Team	2–3 engineers
New language required	No
Primary checker	Dafny
Cross-check	Verus (Weeks 9–12)

5.2 Team Composition

Role	Count	Required Skills
ML Engineer	1	Constrained decoding (XGrammar/Outlines/vLLM), LLM evaluation, prompt engineering
PL/Verification Engineer	1	Dafny fluency, formal methods, transpiler construction
Full-Stack Engineer	1	Building realistic benchmark tasks (REST APIs, DB calls, payment stubs)

5.3 Infrastructure

- GPU access for local model inference (1× A100 or equivalent)

- Dafny toolchain installed
- Benchmark repository (GitHub, private until publication)

5.4 Sub-Experiment A: Surface Syntax Hypothesis

Question: Does a Python/TS-shaped surface syntax + elaboration outperform direct formal-syntax generation?

Week 1–2: Define `ward0` Surface Subset Strict, tiny grammar unambiguously translatable to Dafny. Use PEG grammar (Python `lark` or Rust `pest`) or constrained JSON schema.

`ward0` Example:

```
fn transfer(from: Account, to: Account, amount: int) -> Result<Unit, str>
  requires amount > 0
  requires from.balance >= amount
  ensures from.balance == old(from.balance) - amount
  ensures to.balance == old(to.balance) + amount
{
  from.balance -= amount
  to.balance += amount
  return Ok(())
}
```

`ward0` Rules:

- Only `fn` definitions with `requires` / `ensures` / `old()`
- Primitive types: `int`, `bool`, `str`, `Unit`, `Result<T, E>`, `List<T>`
- Control flow: `if/else`, `for` (bounded loops only), `return`
- No classes, no closures, no generics beyond `Result` and `List`
- All functions pure (no heap mutation) to avoid Dafny framing complexity

Week 2–3: Build the Elaborator Deterministic transpiler (`ward0` $\rightarrow$ Dafny), ~500–1500 lines. Syntax-directed translation.

Example Mapping:

- `ward0 fn` $\rightarrow$ Dafny method or function
- `requires` / `ensures` $\rightarrow$ Dafny `requires` / `ensures`
- `old(x)` $\rightarrow$ Dafny `old(x)`
- `Result<T, E>` $\rightarrow$ Dafny `Result<T, E>` (or hand-rolled datatype)
- `from.balance -= amount` $\rightarrow$ Dafny `from.balance := from.balance - amount;`

Hard constraint: If transpiler cannot translate, throw hard error.

Week 3–4: Build Evaluation Harness **Benchmark:** 50–80 function-level tasks, split into tiers:

Tier	Description	Count	Example
Trivial	Pure arithmetic, array search	15	max_of_list, is_sorted
Standard	Business logic with contracts	30	transfer, validate_email, apply_discount
Tricky	Requires loop invariants or non-trivial postconditions	20-35	binary_search, run_length_encode

Sources:

- DafnyBench (or reconstructed similar tasks)
- HumanEval functions augmented with `requires/ensures`
- Custom full-stack-inspired pure functions

Critical: Hold out 20% for final evaluation. Do not iterate on them during development.

Generation Conditions:

1. **Direct-Dafny (Baseline):** Model generates raw Dafny syntax.
2. **Ward-Surface (Treatment):** Model generates `ward0` syntax $\rightarrow$ transpiler $\rightarrow$ Dafny check.

Model: Single strong open-weight model for both conditions:

- Primary: **Qwen2.5-Coder-32B** or **DeepSeek-Coder-V2-Lite**
- Cross-check: **Claude 3.5 Sonnet** via API (if budget allows)

Generation Setup:

- Temperature = 0.7
- `pass@1`: one shot, no repair loop
- `pass@k`: generate `k=5`, count if any pass
- Ward-Surface only: **grammar-constrained decoding** (XGrammar/Outlines)

Week 5–7: Run Experiment A Generate solutions for all tasks in both conditions. Run Dafny verifier. Record:

- Syntax-error rate
- Verification-pass rate
- Functional correctness (hidden tests against verified code)
- Time-to-generate, time-to-verify

Week 8: Analyze A **Statistical test:** McNemar’s test or paired proportions test on `pass@1`. Need statistically significant improvement in Ward-Surface to proceed.

5.5 Sub-Experiment B: FFI Boundary Hypothesis

Question: Does a contract-stub trust boundary catch more errors at library call sites than unverified direct calls?

Week 4–5: Design Benchmark Tasks 6–10 realistic full-stack scenarios:

Task	Libraries Involved	What to Verify
Payment charge	Stripe SDK stub	Amount > 0, idempotency key handling
User auth	bcrypt / JWT stub	Password hash comparison, token expiry
DB query	SQLite/Postgres stub	SQL injection prevention
REST endpoint	HTTP router stub	Request validation, response shape
Currency conversion	Exchange-rate API stub	Rate freshness, precision handling

Important: Not calling real libraries. Building **stubs** that simulate unverified behavior — including *buggy* behavior.

Week 5–6: Build Stubs and Boundary Two Conditions:

1. **Unverified Baseline:** Code calls stubs directly with no contracts. Hidden tests check for bugs.
2. **Trust-Boundary (Treatment):** Code must declare extern stubs with `requires/ensures`:

```
extern method Stripe_Charge(amount: int, token: string) returns (r: Result<Charge, string>)
  requires amount > 0
  requires |token| > 0
  ensures r.Ok? ==> r.value.amount == amount
```

Error Injection: In 30–50% of hidden tests, stub violates its declared contract deliberately. Question: does the boundary catch the *caller's* misuse even if the library is buggy?

Week 7–8: Run Experiment B

- Model generates caller code and stub contracts.
- Run hidden tests.
- Measure:
 - Contract violation catch rate
 - False positive rate
 - Bug escape rate

5.6 Week 9–10: Integration, Cross-Checks, Write-Up

- Re-run held-out tasks with frozen prompts.
- Sanity-check for prompt leakage or contamination.
- Draft technical report (workshop-paper length: 6–10 pages).
- Prepare go/no-go presentation.

5.7 Week 11–12: Buffer & Go/No-Go Decision

6. Go / No-Go Criteria (Hard Gates)

Gate	Criterion	Threshold
A1	Ward-Surface pass@1 > Direct-Dafny pass@1	Statistically significant ($p < 0.05$) improvement of 5 absolute percentage points
A2	Grammar-constrained decoding eliminates syntax errors	Ward-Surface syntax-error rate <2% (vs. Direct-Dafny expected ~10–20%)
B1	Trust boundary catches caller errors	30% relative reduction in runtime bugs vs. unverified baseline
B2	False positive rate acceptable	15% of valid caller implementations rejected due to overly strict contracts
A3/B3	Functional correctness beats verification-only	Hidden-test pass rate improves proportionally with verification rate (catches vacuous verification)

Proceed to Phase 1 only if **A1**, **A2**, and at least one of **B1/B2** are met.

- If **A1 fails**: Core hypothesis (surface syntax helps) is falsified.
- If **B1 fails but B2 passes**: FFI boundary needs redesign before Phase 1.
- If **both B1 and B2 fail**: Gradual-verification approach needs rethinking.

7. Deliverables at End of Phase 0

1. **Reproducible benchmark suite** (50–80 tasks + 6–10 FFI scenarios), open-sourced or documented.
2. **Transpiler codebase** (ward0 → Dafny), ~1K lines, with tests.
3. **Evaluation harness** with constrained decoding integration.
4. **Technical report** (6–10 pages) with all numbers, including negative results.
5. **Go/No-Go decision document** with explicit recommendation for Phase 1, pivot to Verus-extension, or halt.

8. Immediate Actions (This Week)

Day	Action	Owner
1	Confirm Dafny as baseline checker; finalize team roles	Team Lead

Table 10 – continued

Day	Action	Owner
2	Set up repository + benchmark skeleton	Full-Stack Engineer
3	Install Dafny + XGrammar/ Outlines + inference stack	ML Engineer
4	Draft <code>ward0</code> grammar (v0.1)	PL Engineer
5	Select first 10 benchmark tasks	All
7	Internal review of <code>ward0</code> grammar + task list	All

9. Full Implementation Roadmap (For Reference)

Phase 0: Minimal Falsification Experiment (~3 months, 2–3 engineers)

Test untested claims before building a language. **CURRENT PHASE.**

Phase 1: Core Calculus & Checker (9–12 months, 3–5 PL researchers)

Formalize and implement checker for restricted core: refinement types, effect tracking, linear types. Target decidability and acceptable latency.

- **Go/No-Go:** Verify real program patterns within single-digit seconds per function.

Phase 2: Elaborator & Repair Loop (6–9 months, 2–4 compiler engineers; parallel with late Phase 1)

Build deterministic surface-to-core compiler, structured-error translation, full generate→check→repair pipeline.

- **Go/No-Go:** Converged accuracy exceeds Phase 0 single-shot by margin justifying complexity.

Phase 3: Content-Addressed Composition Library (6 months, 2–3 systems engineers; parallel with Phase 2)

Build proof-caching, content-addressed storage, search-and-compose behavior.

- **Go/No-Go:** Measurable reduction in novel-code surface area without accuracy regression.

Phase 4: Bootstrapping Model Fluency (12–24 months, 5–8 ML/RL engineers; fund only if Phases 0–3 pass)

Generate synthetic corpora, fine-tune/RL against checker as reward signal. **Highest-risk phase.**

- **Go/No-Go:** This phase should not be funded until Phases 0–3 are complete and positive.

10. Risk Register

Risk	Severity	Mitigation
Full dependent-type validity is undecidable	High	Scope Phase 1–2 to decidable subset (refinement types à la LiquidHaskell).
Totality checkers are conservative/imperfect	Medium	Budget for friction; measure false-positive rejection rate in Phase 1.
Checker-driven RL may not generalize to general software (H3)	High —central unproven claim	Sequence last (Phase 4). Phases 0–3 produce value independent of H3.
Type-constrained decoding’s 74.8% figure measures compile errors, not functional correctness	Medium	Phase 0 success criteria defined on functional correctness (pass@1), not compile rate.
Gradual Verification untested for high-churn web libraries	Medium	Explicitly scoped as open question in Phase 0-B, not assumed to transfer.
New-language bootstrapping: zero model fluency on day one	High for later phases	Phase 0 requires no new language and no training.
Multi-target compilation trust gap	Medium —acknowledged	Proven-tier code targets only deterministic backends (native, WASM). Never target probabilistic translation steps.
Build-vs-extend: Verus may be cheaper path	High strategic risk	Phase 0.5 spike (Weeks 9–12) answers this before Phase 1 commitment.

11. Evaluation & Success Metrics

Primary Metric (all phases): Functional correctness (pass@1) on:

- Held-out, contamination-checked slice of SWE-bench Pro
- GitChameleon (library-version compatibility)
- Custom Ward regression suite (checker soundness, false-positive rate)

Secondary Metric (tracked only to prevent substitution): Compile-error / verification-pass rate.

Definition of Success: Niche, institutional production adoption on the scale of Dafny, SPARK, or Verus (~2,700–8,500 GitHub stars, real institutional users). **Not** mass/Rust-scale adoption.

First Exposure Milestone: Phase 0 results constitute a legitimate submission to the auto-active verification workshop (co-located with POPL), regardless of outcome.

12. Positioning Against Existing Languages

Capability	Dafny	Lean 4	SPARK/Ada	Verus	Ward (Proposed)
Contracts + SMT verification	Yes, native	Via extensions	Yes, native	Yes, native	Yes, native
Published AI self-healing success	82–91%	Strong (proofs)	50.7%; 49,280 obligations	Emerging	Untested (Phase 0 target)
Familiar surface syntax	No (~77% pass@1 ceiling)	No	No	Yes (Rust)	Yes (Python/TS-shaped)
Multi-target / real ecosystem	Yes (C#/Java/JS/Go/Python)	No	Limited (~400 packages)	Yes (inherits crates.io)	Proposed (JS/TS, Go, Python, JVM)
Tiered / gradual verification	No	No	Contracts optional	No	Yes (Tested/Contracted/Proven)
Trusted boundary for unverified libraries	No native	No native	No native	No native	Proposed (applying Gradual Verification)
Full-stack effect vocabulary	No (heap-only)	No	No	No	Proposed
Reusable contract templates	No	No	No	No	Proposed (expand-then-recheck)
Community size (GitHub stars)	3,400	8,500	Industrial/commercial	~2,680	0 —unbuilt
Production use today	Yes (AWS)	Yes (formal math)	Yes (aerospace/defense)	Emerging	None —proposed only

What this table shows: Every row where Ward has an entry and others don't is a **proposed capability, not demonstrated**. The other four columns reflect years of real engineering. Ward's case is that these specific gaps are real and shared across all four existing languages.

Document compiled from Ward Language Design (ward-language-design.md) and Ward Research Proposal (ward-research-proposal.docx) with planning conversations. All claims trace to primary or credible sources per the fact-check logs in the original design document.